AI LAB TEST-1

ROLL NO:2503A51L09

NAME:D.ANKITHA

BATCH:19

Scenario: Suppose you are building an AI-powered chatbot to assist students in answering questions related to their coursework. To make it effective, you must optimize prompts and manage contextual information efficiently.

Tasks 1

Use GitHub Copilot to generate different prompt variations for the chatbot (e.g., answering short factual queries vs. open-ended essay-style questions).

code:

```python
def get_factual_qna():
    qna_pairs = {
        "What is the largest planet in our solar system?": "Jupiter",
        "Who wrote Romeo and Juliet?": "William Shakespeare",
        "When was the World Wide Web invented?": "1989",
        "What is the capital of Japan?": "Tokyo",
        "What is the boiling point of water?": "100°C at standard atmospheric pressure"
    }
    return qna_pairs

# Example usage:
for question, answer in get_factual_qna().items():
    print(f"Q: {question}")
    print(f"A: {answer}\n")
# This function returns a dictionary of factual question-and-answer pairs.
```

output:

```
Q: What is the largest planet in our solar system?
A: Jupiter

Q: Who wrote Romeo and Juliet?
A: William Shakespeare

Q: When was the World Wide Web invented?
A: 1989

Q: What is the capital of Japan?
A: Tokyo
```

TASK#2:

PROMPT:

Implement a Python script that stores previous user–AI interactions in a conversation
history (e.g., using a list or dictionary).
Use Copilot suggestions to integrate this context so the chatbot can give more relevant answers

CODE:

```
> Users > saian > OneDrive > Documents > Desktop > TASK 0222.py > ...
import time

# List of toxic or biased keywords to filter
toxic_keywords = [
    "hate", "kill", "racist", "violence", "terror", "abuse", "sexist", "discriminate"
]

# List of sensitive topics that require disclaimers
sensitive_topics = {
    "mental health": "⚠ Disclaimer: This response is for informational purposes only and not a substitute for professional advice.",
    "politics": "⚠ Disclaimer: Political views vary widely. This response does not reflect any endorsement.",
    "religion": "⚠ Disclaimer: Religious beliefs are deeply personal. This response aims to be respectful and neutral.",
    "medical": "⚠ Disclaimer: This is not medical advice. Please consult a healthcare professional for accurate information."
}

# Conversation history list
conversation_history = []

def is_toxic(message):
    """Check for toxic or biased content."""
    message_lower = message.lower()
    return any(word in message_lower for word in toxic_keywords)

def get_disclaimer(message):
    """Return disclaimer if message contains sensitive topic."""
    message_lower = message.lower()
    for topic, disclaimer in sensitive_topics.items():
        if topic in message_lower:
            return disclaimer
    return None

def generate_ai_response(user_message):
    """Generate AI response with safeguards."""
    if is_toxic(user_message):
        return "⚠ Your message contains inappropriate or harmful language and cannot be processed."

    disclaimer = get_disclaimer(user_message)
    base_response = "Thank you for your question. Here's some information that may help you."

    if disclaimer:
        return f"{disclaimer}\n{base_response}"
    else:
        return base_response

```

OUTPUT:

```
Open-Ended Essay Question:
Discuss the impact of technology on modern education.

Open-Ended Essay Question:
Explain the causes and effects of climate change in detail.

Open-Ended Essay Question:
Write an essay on the importance of teamwork in professional environments.
```

TASK#3

PROMPT:

Add code-level safeguards (with Copilot assistance) to ensure that the system avoids biased or
harmful responses (e.g., filtering toxic content, adding disclaimers).
• Document on how responsible AI practices were integrated into your code.

CODE:

```
        return f"{disclaimer}\n{base_response}"
    else:
        return base_response

def chat():
    print("🤖 Welcome to the Safe AI Chatbot! Type 'exit' to end the conversation.\n")

    while True:
        user_input = input("👤 You: ")
        if user_input.lower() == "exit":
            break

        timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
        ai_response = generate_ai_response(user_input)

        # Store interaction
        conversation_history.append({
            "Time": timestamp,
            "User": user_input,
            "AI": ai_response
        })

        print(f"🤖 AI: {ai_response}\n")

    # Display full conversation history
    print("\n📂 Conversation History:")
    for i, exchange in enumerate(conversation_history, start=1):
        print(f"\nInteraction {i} [{exchange['Time']}]:")
        print(f"👤 User: {exchange['User']}")
        print(f"🤖 AI: {exchange['AI']}")

# Run the chatbot
chat()
```

output:

```
You: hi
[AI] You said: 'hi'. I'm here to help!
You: & C:/Users/Dell/anaconda3/python.exe "c:/Users/Dell/Desktop/lab exam/shashi34.py"
[AI] You said: '& C:/Users/Dell/anaconda3/python.exe "c:/Users/Dell/Desktop/lab exam/shashi34.py"'. I'm here to h
elp!
You: name
[AI] You said: 'name'. I'm here to help!
You: you from
[AI] You said: 'you from'. I'm here to help!
You:
```